

Beat 4. What this unlocks (6 min)

You have written the thing that makes a list a list. Everything that grows works this way: ask for more room, copy across, release the old, repoint. Python's `list` does it, and so does C++'s `vector`.

Friday: the array is still out there after the object is gone.

CS112: Wednesday 30 September, 12:15 to 1:20

Monday we wrote `PyList` together. A class holding `Item *myArray`, `mySize` and `myCapacity`; a constructor that sets the pointer to `nullptr`; an `append` that makes room for one more item every time it is called. It compiles, it passes its test, and every line in it is correct.

Today: the object is not the data. `myArray` holds an address, the items sit somewhere else, and that address moves every time the array grows. Once that is on the board, our `append` turns out to be doing far more work than it needs to.

TIME	BEAT
12:15	Daily quiz
12:20	1. What does our <code>append</code> cost?
12:34	2. Make it cost less, and ? :
12:56	3. The debugger, on our own class
13:14	4. What this unlocks

Beat 1. What does our `append` cost? (14 min)

Open `PyList.cpp`. Run `make test`. It passes, as expected. Nothing here is broken. Now let's think about what these operations cost.

Talk to your neighbor. Every `append` asks for a new array and copies everything across. How many item copies is that, for 100 appends?

Answers first, confirmation after. The first `append` copies nothing, the second copies 1, the third copies 2, and the hundredth copies 99. The sum goes on the board:

$$\text{copies} = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{100 \times 99}{2} = 4950$$

The closed form is the point, not the number: **n^2 over 2**, so ten times the appends is a hundred times the copying. Measured on our own file: **4,950 item copies and 100 allocations**.

Then the problem goes back to the room, not the answer:

Talk to your neighbor. The copying is what costs. How could we copy less often?

Whatever comes back gets costed on the board.

STRATEGY	100 APPENDS	1000 APPENDS
one more slot each time	4,950 copies	499,500 copies
ten more slots each time	450 copies	49,500 copies
twice the room each time	127 copies	1,023 copies

Ten extra slots is a real improvement and still the wrong shape: ten times the appends, a hundred times the copying. Only the doubling row barely moves.

Beat 2. Make it cost less, and `? :` (22 min)

Live coding, starting from the condition. When does the array need to grow? `mySize == myCapacity`, from the room.

```
void PyList::append(Item it) {
    if (mySize == myCapacity) { // full; empty is full too
        int cap;
        if (myCapacity == 0) cap = 1;
        else cap = myCapacity * 2;
        Item *bigger = new Item[cap];
        for (int i = 0; i < mySize; ++i)
            bigger[i] = myArray[i];
        delete [] myArray;
        myArray = bigger;
        myCapacity = cap;
    }
    myArray[mySize] = it;
    ++mySize;
}
```

The empty list was never a special case: it is an array with no room left, which the one condition already catches.

The four lines that choose `cap` then collapse into one:

```
int cap = (myCapacity == 0) ? 1 : myCapacity * 2;
```

Read aloud: "if this, then that, otherwise the other".

`if` is a **statement**: it *does* something. `? :` is an **expression**: it *is* a value. That is why it can sit on the right of an `=` and the `if` cannot.

Three rules: choose **between two values for one variable**, never nest them, no side effects inside. Then the trap, on screen:

```
cout << x > 0 ? "positive" : "negative";
```

```
error: no match for 'operator>' ('std::basic_ostream<char>' and 'int')
```

`<<` binds tighter, so the compiler read `(cout << x) > 0`. Parenthesise it inside any larger expression.

Monday's test runs again and still passes: after two appends the capacity is still 2, because doubling goes 0, 1, 2. **The test cannot tell the two versions apart.**

What does a passing test prove, and what does it not?

Beat 3. The debugger, on our own class (18 min)

Set up on `PyList` on the projector, the room following on their own machines.

1. **Breakpoint.** Gutter left of the line number, red dot, on `int cap = ...`.
2. **F5**, pick the configuration. It stops on the dot.
3. **Variables.** Expand `this`; add `myArray` to **Watch**.
4. **F10** over, **F11** into, **F5** continue.

Append nine items. Before each F5: will it stop again? Will the address change?

Monday's version stops on every append, with a new address every time. Today's stops five times: capacity 1, 2, 4, 8, 16.

Why do the stops get further apart?

The array moves, and anything holding the old address is holding a lie.